

Data Analytics para el Control de Gestión

Python para Control de Gestión

César Núñez Cuevas
cnunez28@santotomas.cl



- Corresponde al intento de establecer relaciones entre distintas estructuras tabulares. Esto permite generar bases de datos o DataFrames más grandes a través de la unión de fuentes relacionadas.
- A diferencia de una simple concatenación de filas o columnas, en análisis de datos con Python buscamos realizar uniones relacionales (*joins*) basadas en llaves o identificadores comunes.

Uniones: Llave Primaria y Foránea

- En análisis de datos con Pandas, las columnas de unión actúan como nexos lógicos. Buscamos métodos que mantengan la integridad y correspondencia exacta de la tabla principal.
- Consideremos dos bases, donde **A** contiene la llave primaria y **B** contiene la llave foránea:

A	B
1	3
2	4
3	5
4	6

Uniones: Llave Primaria y Foránea

- Una **llave primaria** (Primary Key) corresponde a la columna de la base a la cual deseamos añadir información, donde cada valor debe ser único.
- Una complejidad común al programar en Python o SQL es que las llaves primarias no deben tener valores duplicados, de lo contrario Pandas multiplicará las filas inesperadamente (creando productos cartesianos).
- Por el contrario, en hojas de cálculo tradicionales (como Excel) al buscar celda por celda esta restricción estructural no se valida automáticamente.

Validación de Llaves y Duplicados en Pandas

Antes de ejecutar un `pd.merge()`, es fundamental verificar la integridad de las llaves para evitar errores analíticos:

- **Verificación de unicidad:** Podemos comprobar si una llave primaria tiene duplicados usando el método `is_unique`
- Ejemplo de código:

```
if not df_A['id_primary'].is_unique:  
    print("¡Advertencia: Hay duplicados en la llave primaria!")
```
- Consecuencia: Si una llave primaria está duplicada en ambas tablas, Pandas generará una combinación de todos con todos (explosión de filas).

Tipos de Joins en Pandas

- **Inner Join:** Une las tablas basándose estrictamente en la intersección de las llaves en ambas fuentes (`how='inner'`).

A	B
3	3
4	4

- **Outer Join (Full):** Combina todas las filas de A y de B. Las celdas que no tengan correspondencia generarán valores nulos (`NaN`) (`how='outer'`).

A	B
1	NULL
2	NULL
3	3
4	4
NULL	5
NULL	6

Outer Joins en Pandas

- **Right Outer Join:** Predominan todas las filas de la tabla B, trayendo los datos coincidentes de A (`how='right'`).

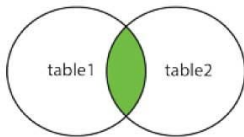
A	B
3	3
4	4
NULL	5
NULL	6

- **Left Outer Join:** Predominan todas las filas de la tabla A, trayendo los datos coincidentes de B (`how='left'`).

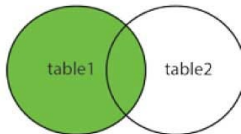
A	B
1	NULL
2	NULL
3	3
4	4

Tipos de Joins en Pandas

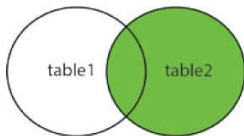
INNER JOIN



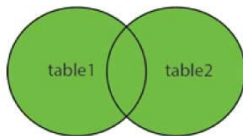
LEFT JOIN



RIGHT JOIN



FULL OUTER JOIN



Manejo de Conflictos por Nombres de Columnas

Cuando dos DataFrames que vamos a unir poseen columnas con el mismo nombre (ej. fecha o descripcion), Pandas aplica sufijos automáticamente:

- **Parámetro suffixes:** Permite renombrar las columnas repetidas para conservar el origen de los datos.

- **Sintaxis en Pandas:**

```
pd.merge(df_A, df_B, on='id_vuelo',  
         suffixes=('_ori', '_dest'))
```

- **Buenas prácticas:** Renombrar explícitamente las columnas antes del cruce si los nombres ambiguos pueden confundir en etapas posteriores del análisis (*feature engineering*).

¿Por qué organizar tablas? De Joins a Modelos Relacionales

Estructurar múltiples DataFrames mediante relaciones relacionales (*Joins*) aporta grandes ventajas en Python:

- **Eficiencia:** Evita duplicar textos largos o datos redundantes en una sola tabla gigante.
- **Filtros Cruzados:** Permite aislar tablas de dimensiones para filtrar métricas de forma limpia.
- **Rendimiento en Memoria:** Pandas procesa y opera mucho más rápido con múltiples tablas optimizadas que con una mega-tabla plana llena de duplicados.

El objetivo

Crear una estructura limpia donde los hechos (métricas numéricas) y las dimensiones (contexto) se unan solo cuando el análisis lo requiera mediante `pd.merge()`.

Esquema en Estrella (Star Schema en Python)

Es el modelo más sencillo y eficiente para estructurar proyectos analíticos en Pandas.

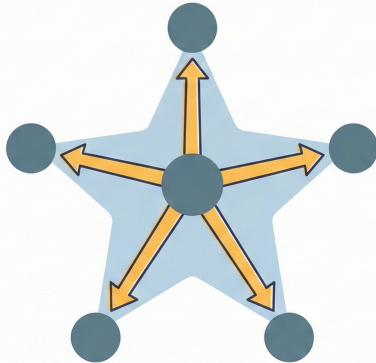
- **Tabla de Hechos (Fact):** Al centro. Contiene datos cuantitativos y transaccionales (ej. Vuelos, Costos).
- **Tablas de Dimensiones:** Alrededor. Contienen descriptores (Aeropuertos, Aviones, Rutas).
- **Relación:** Uniones limpias de 1 a Muchos mediante llaves foráneas.

Beneficios:

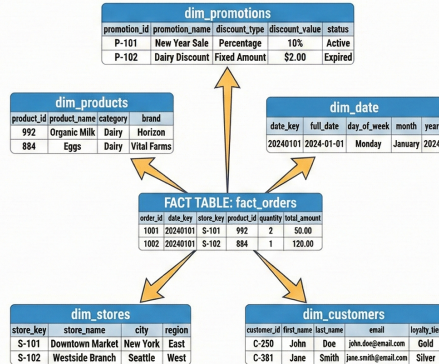
Simplicidad

Facilita las agrupaciones (*groupby*) y evita cruces confusos de múltiples tablas al mismo tiempo.

STAR SCHEMA HUB AND SPOKE MODEL



ACTUAL STAR SCHEMA WITH DATA



Optimización de Memoria en Esquemas Estrella

Al trabajar con bases de datos grandes en Python, el tipo de datos influye drásticamente en el consumo de RAM:

- **Tipos Categóricos:** Las columnas de dimensiones repetidas (ej. nombre de aerolíneas, regiones) consumen mucha memoria si se guardan como object (strings).
- **Conversión:** Se recomienda transformar las llaves foráneas y atributos descriptivos de baja cardinalidad a tipo category:

```
df['aerolinea'] = df['aerolinea'].astype('category')
```
- **Impacto:** Reduce el peso del DataFrame en memoria hasta en un 80 %, acelerando drásticamente las operaciones de merge y groupby.

Esquema en Copo de Nieve

Es una variante donde las tablas de dimensiones se normalizan dividiéndose en subtablas adicionales.

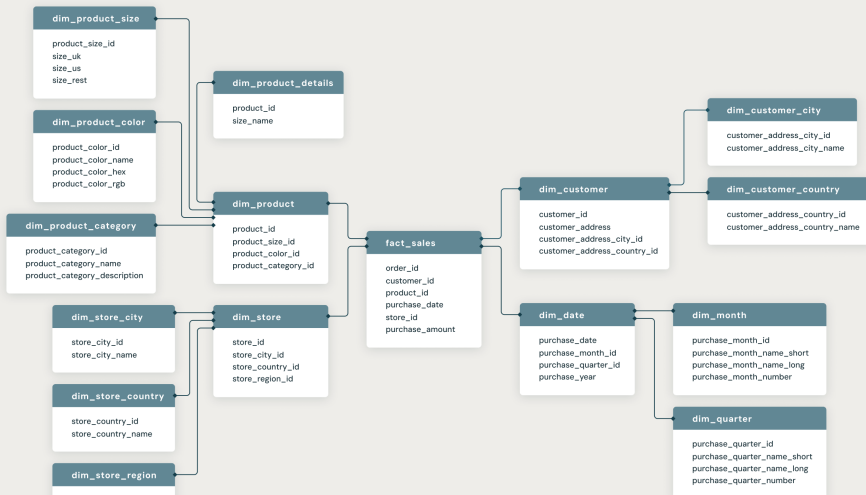
- **Estructura:** Las dimensiones no conectan en un solo nivel con los hechos (ej. Hechos → Ruta → Aeropuerto de Origen).
- **Normalización:** Reduce el espacio en memoria al evitar repetir descripciones encadenadas.
- **Complejidad en Python:** Requiere realizar múltiples `pd.merge()` sucesivos para consolidar la información final.

Visualización

Se llama así porque el diagrama ramificado se asemeja a un cristal de nieve.

Esquema en Copo de Nieve

Snowflake Schema



Diferencias Clave: Estrella vs. Copo de Nieve

Criterio	Estrella	Copo de Nieve
Rendimiento en Python	Alto (Pocos cruces merge)	Menor (Múltiples merge enca- denados)
Mantenimiento de código	Sencillo y directo	Complejo (Muchas tablas in- termedias)
Uso en Análisis	Estándar recomendado en Pandas	Común en bases de datos nor- malizadas
Estructura	Desnormalizado (Más plano)	Altamente normalizado

¿Cuál elegir en tus scripts?

- 1 **Usa Esquema en Estrella:** Siempre que sea posible para simplificar el código de tus análisis y acelerar los cálculos.
- 2 **Usa Copo de Nieve:** Únicamente cuando el dataset original provenga de una base de datos relacional altamente normalizada y debas mantener esa estructura por requerimientos del proyecto.

- **Flights:** Contiene los identificadores de vuelos, métricas de operación y llaves hacia rutas y aeronaves.
- **Routes:** Rutas de vuelo detalladas.
- **Aircraft:** Tipos y especificaciones de aviones.
- **Airports:** Set de datos descriptor con información geográfica de aeropuertos en EE. UU.

Alternativas a Merge: `join()` y `concat()`

Además de `pd.merge()`, Pandas ofrece otras herramientas para combinar estructuras de datos:

- `DataFrame.join()`:
 - Está diseñado para unir tablas basándose en sus **índices** (Index) en lugar de columnas arbitrarias.
 - Su sintaxis es más limpia cuando trabajamos con índices jerárquicos.
- `pd.concat()`:
 - Apila DataFrames verticalmente (*axis=0*) u horizontalmente (*axis=1*).
 - Se utiliza para concatenar tablas con la misma estructura (ej. unir reportes mensuales de ventas), no para relaciones relacionales.

Preprocesamiento y Exploración en Python

Se suele pensar que importar datos es solo cargarlos en un DataFrame. Sin embargo, este es el primer paso vital para inspeccionar la estructura y comprender la lógica analítica:

- **¿Cuáles son las variables de cada set de datos?:** Analizar el significado de cada columna (ej. FuelCostperSeatMile).
- **¿Se observa algún error en los datos?:** Validar tipos de datos, fechas y nulos.
- **¿Posee la estructura correcta?:** Comprobar que las filas representen observaciones únicas y las columnas variables independientes, sin celdas combinadas.
- **¿Existen identificadores únicos?:** Verificar la existencia de llaves primarias válidas para realizar los futuros *Joins*.

Buenas Prácticas y Depuración Post-Merge

Una vez finalizado el cruce de tablas, es crítico realizar controles de calidad sobre el DataFrame resultante:

- **Control de conteo de filas:** Comprobar que el tamaño del DataFrame tras un *inner join* o *left join* coincida con las expectativas teóricas.

- **Detección de nulos post-uniión:**

```
df_resultado.isna().sum()
```

Ayuda a identificar si hubo llaves foráneas en la tabla de hechos que no encontraron coincidencia en las dimensiones (registros huérfanos).

- **Limpieza de llaves auxiliares:** Eliminar columnas de ID redundantes que ya no se utilizarán en la etapa de modelado o visualización.